SRU

SR UNIVERSITY

Department of Computer Science & Engineering [AIML]

Batch – 9

JOBSPACE

A Unified Job Application Intelligence Platform

A Technical Research Paper

Technology Stack:

Node.js • Express.js 5 • MongoDB Atlas • Mongoose • JWT • React • Vercel • Render

2303A52325 KANDAGATLA SAI TEJA

2303A52330 BANDI RITHWIK

2303A52352 ELIKATTE THARUN

2303A52356 KUMBA GANESH

April 2026

Abstract

The modern job search is a cognitively and operationally demanding process in which candidates simultaneously manage dozens of open applications, maintain structured recruiter communications, version multiple resume documents, and track companies at varying stages of interest, yet no purpose-built, engineering-grade candidate-facing platform exists to unify these workflows under a single intelligent system. JobSpace addresses this gap by delivering a full-stack, multi-tenant web application that integrates job application lifecycle management, company CRM intelligence tracking, document versioning, and a behavioural analytics engine into one cohesive platform.

The system is built upon a decoupled three-tier architecture: a React single-page application (SPA) frontend deployed on the Vercel CDN communicates exclusively through a RESTful API built on Express.js 5 (Node.js 20), which is hosted on Render and backed by MongoDB Atlas. A key engineering contribution is the dual-mode authentication system that resolves user identity through either a JWT Bearer token (for cross-domain production deployments) or a session-cookie fallback (for development environments), addressing a well-documented deployment pain-point in SPA architectures. A second major innovation is the dynamic column engine implemented via the Spreadsheet-Column Mongoose model, which allows each user to define, type, reorder, and persist arbitrary spreadsheet columns (text, number, date, and dropdown with colour-coded labels) against both their applications and companies tables without any database schema migration.

The analytics subsystem leverages MongoDB's aggregation framework to compute multi-dimensional job search metrics, including application-to-interview response rates, offer conversion funnels, monthly application velocity, and company targeting distributions, via six parallel aggregation pipelines executed concurrently through `Promise.all()`. This paper presents the full system architecture, data-modelling decisions, API design, security mechanisms, performance characteristics, and future research directions of JobSpace, providing a developer-grade technical reference for the design decisions made throughout the project.

Keywords: Job Application Tracking, SPA Architecture, MongoDB Aggregation Pipeline, JWT Authentication, Dynamic Schema Engine, Role-Based Access Control, REST API Design, Full-Stack Web Development, Document Management, Cross-Origin Session Management.

Contents

1. Introduction

1.1 Problem Statement

The job search process for a mid-career professional is operationally comparable in complexity to managing a small sales pipeline. A typical candidate applies to between 50 and 200 positions over a search duration of two to six months, maintains ongoing communication with 10 to 30 recruiters across multiple channels, prepares and maintains several versions of their curriculum vitae tailored to different role types, and tracks companies at varying stages of exploratory interest. This process is inherently asynchronous, multi-channel, and stateful, yet the dominant tools employed to manage it remain generic consumer spreadsheets, personal email folders, and fragmented notes applications.

This reliance on unstructured tooling produces compounding operational failures: the absence of structured status tracking causes missed follow-up opportunities and duplicate applications; manual document version management leads to the inadvertent submission of outdated resumes; the lack of quantitative analytics prevents candidates from identifying their strongest application channels, optimal outreach timing, or bottlenecks in their conversion funnel; and the disconnection between company research and the live application pipeline results in the loss of recruiter contact histories and relationship context.

1.2 Industry Context and Motivation

The global recruitment technology market was valued at approximately USD 2.9 billion in 2023 and is projected to grow at a compound annual growth rate of 7.4 percent through 2030 [1]. However, this market is almost entirely oriented toward the employer side of the hiring process: Applicant Tracking Systems (ATS), sourcing intelligence platforms, and HR analytics tools serve the recruiter, not the candidate. The candidate-facing tooling segment remains a largely unsolved and commercially underserved problem space.

Existing candidate-side tools such as Huntr, Teal HQ, and Simplify offer basic Kanban-style boards with fixed, hardcoded column sets and rudimentary status tracking. None of these systems provides the engineering depth required for a full analytics pipeline, per-user dynamic schema customisation, or an authentication architecture designed for the realities of cross-domain cloud deployments. The academic and engineering literature similarly lacks a formal treatment of candidate-facing job search intelligence systems designed with production-grade software architecture principles.

1.3 Contributions

This paper makes the following technical contributions:

- A production-deployed, multi-tenant full-stack web application for job application lifecycle management, encompassing six integrated functional modules.
- A dual-mode authentication architecture that resolves user identity through JWT Bearer tokens for cross-domain production use and session-cookie fallback for development, from a single shared middleware implementation.
- A dynamic column engine that allows per-user, per-page spreadsheet schema customisation, including typed columns, ordered layouts, and colour-coded dropdown options, without database schema migrations.

- A MongoDB aggregation pipeline analytics engine that computes application funnel metrics, time-series velocity data, and multi-dimensional distributions via six concurrent aggregation queries.
- A formal comparative analysis of JobSpace against existing candidate-facing job tracking tools, identifying the architectural and functional differentiators.

1.4 Paper Organisation

Section 2 surveys the relevant literature and positions this work within the existing landscape. Section 3 presents the proposed system and its key innovations. Section 4 details the system architecture. Section 5 describes the development methodology and technology justification. Section 6 presents the detailed system design, including database schema, API structure, and module breakdown. Section 7 covers the implementation details of the most significant engineering subsystems. Section 8 presents results and system analysis. Section 9 provides a comparative analysis against existing tools. Sections 10, 11, and 12 address limitations, future scope, and conclusion respectively.

2. Literature Survey

2.1 Candidate-Facing Job Tracking Systems

A systematic survey of the current landscape reveals a sharp bifurcation between low-fidelity consumer tools and high-complexity enterprise ATS platforms, with a conspicuous gap at the candidate-facing intelligent tracking tier. Huntr (2023) provides a Kanban board with per-column job cards but offers no analytics pipeline, no document management, no dynamic column customisation, and no API-first design. Teal HQ offers a browser extension for job capture and basic resume building but lacks a CRM-style company tracking module, quantitative analytics, and server-side data processing. Simplify focuses exclusively on auto-fill assistance for job application forms and provides no post-application tracking or analytics whatsoever.

LinkedIn's built-in "Applied Jobs" feature tracks submission history for roles applied to through the LinkedIn platform but is restricted to that single source, provides no status lifecycle management, and exposes no analytics, document vault, or company CRM capability. Consumer spreadsheet tools (Google Sheets, Microsoft Excel) offer maximum flexibility but no structure: they do not enforce data types, cannot compute derived metrics without manual formula construction, provide no authentication or multi-device access control, and require entirely manual maintenance.

2.2 ATS Systems and the Employer–Candidate Asymmetry

Applicant Tracking Systems such as Greenhouse, Lever, and Workday Recruiting are purpose-built for the employer side of the hiring relationship. They manage job postings, parse incoming resumes, schedule interviews, and facilitate team collaboration on candidate evaluation. Breaugh (2013) identifies the information asymmetry inherent in recruitment: employers accumulate structured, queryable data about every candidate who applies to their organisation, while candidates accumulate no corresponding structured data about their own search behaviour, channel performance, or conversion rates [2]. JobSpace is explicitly designed to close this asymmetry from the candidate perspective.

2.3 CRM Methodologies Applied to Job Searching

Several practitioners and researchers have proposed applying Customer Relationship Management (CRM) principles to the job search process, treating each prospective employer as a "lead" in a structured pipeline [3]. This conceptual framing directly informs the Company module in JobSpace, which models employer entities with explicit status lifecycle stages (Dream, Applied, Interviewing, Offer, Rejected), structured recruiter contact fields, and an optional cross-reference to linked Application records. The academic CRM literature, particularly the pipeline management work of Payne and Frow (2005), provides the theoretical grounding for this design [4].

2.4 Dynamic Schema Design in Document Databases

MongoDB's document model inherently supports schemaless storage, but practical application-layer schemas enforced through object-document mappers such as Mongoose constrain this flexibility to protect data integrity. The SpreadsheetColumn approach in JobSpace aligns with patterns described by Sadalage and Fowler (2012) in their treatment of polyglot persistence: a metadata document defines the structure of a flexible data store, while actual values are persisted in a mixed-type or map-type field [5]. This mirrors the Entity–Attribute–Value (EAV) pattern from relational database design, adapted to the MongoDB document model and extended with a rich type system including colour-coded dropdown option sets.

2.5 JWT and Session Authentication in SPA Deployments

The architectural tension between JSON Web Token stateless authentication and server-side session management in single-page application deployments is well-documented in the engineering literature. The OWASP Authentication Cheat Sheet (2023) highlights the core trade-off: JWTs are stateless and cross-domain compatible but cannot be revoked before their expiry without additional infrastructure; server-side sessions are revocable and tamper-evident via HMAC but require shared persistent storage in distributed or multi-instance deployments [6]. Hulse (2020) identifies the cross-domain cookie restriction, where `SameSite` and `Secure` cookie attributes prevent session cookies from being sent on cross-origin requests, as the primary reason SPAs deployed on different domains from their APIs default to JWT-based authentication [7]. JobSpace's dual-mode architecture is a pragmatic hybrid that accommodates both constraints simultaneously, resolving them at the middleware layer without requiring environment-specific code in protected route handlers.

2.6 Identified Research Gap

The literature survey identifies the following gap: no existing system combines candidate-facing job application tracking, company CRM intelligence, document version management, quantitative analytics, and a production-deployed API-first architecture with per-user dynamic schema customisation. JobSpace is positioned as the first engineering-grade candidate intelligence platform that addresses all five dimensions simultaneously, with a formally documented architecture suitable for academic analysis.

3. Proposed System

3.1 System Overview

JobSpace is a multi-tenant, cloud-native web application that manages the complete candidate-side job search lifecycle, from initial wishlist formation through offer acceptance, while providing structured data capture, company relationship intelligence, document version management, and a quantitative analytics engine. The system is architected as a decoupled single-page application communicating with a RESTful backend API, deployed across two cloud services (Vercel for the frontend, Render for the API) with MongoDB Atlas as the cloud database.

3.2 Key Innovations

3.2.1 Dual-Mode Authentication Architecture

The authentication subsystem implements a resolution chain in a single middleware function (`resolveUser`) that first attempts to verify a JWT Bearer token from the Authorization HTTP header, the standard mechanism for cross-domain SPA-to-API communication, and on failure falls back to reading the user identity from a MongoDB-backed `express-session`. On successful JWT resolution, the middleware backfills the session object with the resolved user fields, maintaining backward compatibility with all legacy session-reading code. This design eliminates the need for environment-specific conditional logic in any protected route handler and gracefully supports both the cross-domain production deployment (Vercel to Render) and the same-origin development environment from a single shared codebase.

3.2.2 Dynamic Column Engine via Metadata-Driven Schema

The most architecturally significant innovation in JobSpace is the dynamic column engine. Unlike all competing tools, which provide fixed, hardcoded spreadsheet column sets, JobSpace allows each user to define arbitrary additional columns of four types, namely text, number, date, and dropdown, against both their Applications and Companies tables. Column definitions are persisted in the `SpreadsheetColumn` Mongoose model as typed metadata documents (one per user per page, enforced by a unique compound index). Standard fields are stored in named Mongoose schema properties (flagged `dbField: true`) for indexed querying; custom column values are stored in a MongoDB `Mixed`-type `extraData` field on each Application or Company document. The controller PATCH handler performs runtime field dispatch: incoming keys that match the named field list are applied as direct `$set` operations; keys prefixed with `custom_` or `extra_` are applied via MongoDB dot notation to the `extraData` subdocument. This architecture enables fully dynamic, user-customisable spreadsheet views without any database schema migration, application redeployment, or backend code change.

3.2.3 Parallel Aggregation Pipeline Analytics Engine

The analytics subsystem computes six distinct multi-dimensional metrics from the Application and Document collections using MongoDB's aggregation pipeline framework. All six pipelines are dispatched concurrently via JavaScript's `Promise.all()`, eliminating sequential execution latency. The computed metrics include: application status distribution, top-targeted company frequency, job type breakdown, 12-month application velocity time series, document upload activity time series, and raw total counts. Response rate (interviews divided by applied applications) and offer rate (offers divided by interview-stage applications) are derived arithmetically from the aggregated counts at the application layer. This server-side computation strategy minimises API response payload size for users with large application histories, as only derived summary data, not raw application arrays,

is transmitted to the client.

4. System Architecture

4.1 High-Level Architecture

JobSpace employs a classical three-tier architecture: a Presentation Tier (React SPA on Vercel CDN), an Application Tier (Express.js 5 REST API on Render), and a Data Tier (MongoDB Atlas cloud cluster). The Presentation Tier communicates with the Application Tier exclusively through HTTPS REST calls, carrying a JWT Bearer token in the Authorization header for all authenticated requests. The Application Tier communicates with the Data Tier through the Mongoose ODM over a TLS-encrypted MongoDB Atlas connection string.

The system does not employ a backend-for-frontend (BFF) intermediary or a GraphQL gateway; the REST API is consumed directly by the SPA client. Static file serving for uploaded documents is handled by Express's built-in static middleware, exposing files uploaded via Multer from the `/uploads` directory at the `/uploads` URL prefix. Session state is stored in MongoDB via `connect-mongo`, co-located with application data on the same Atlas cluster.

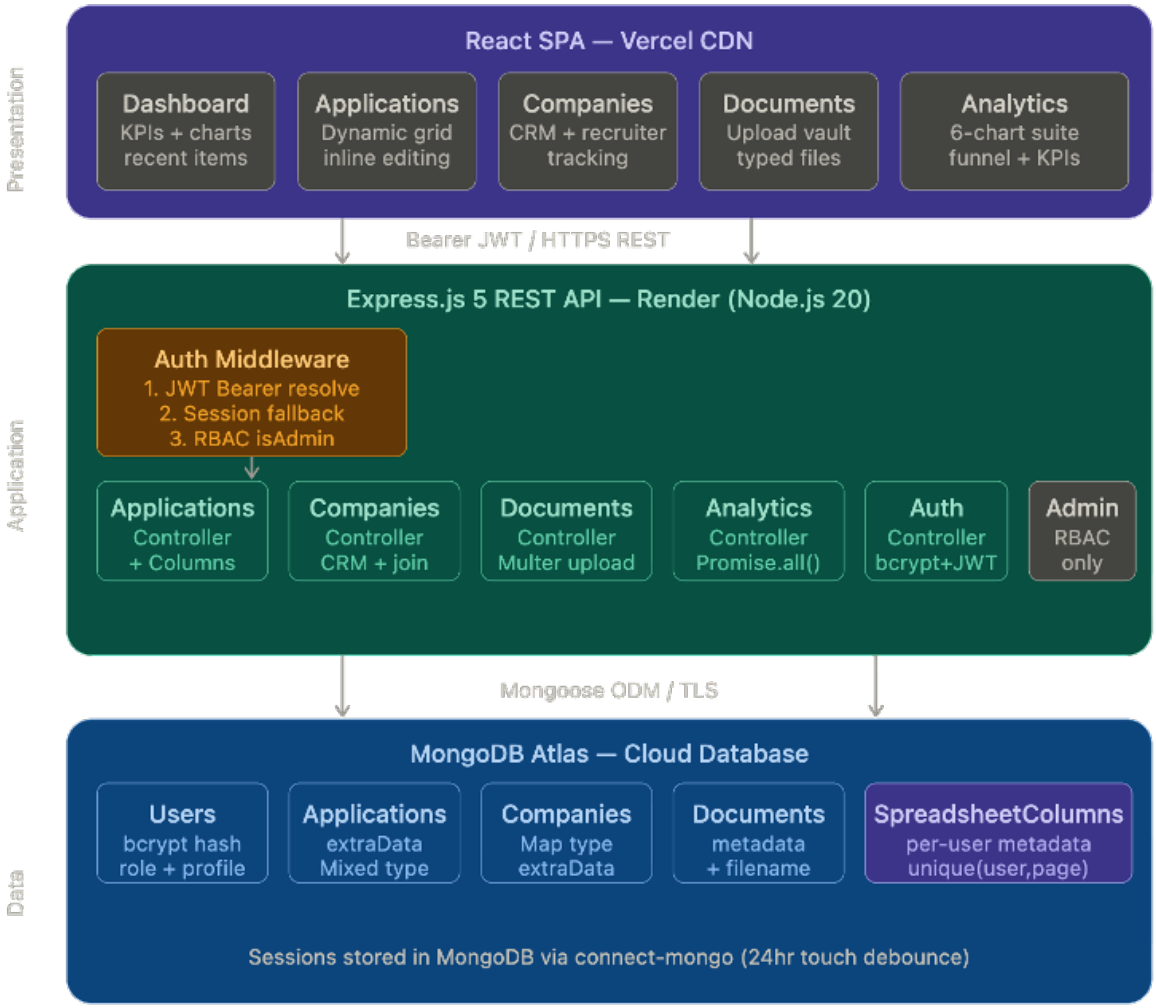

Figure 1: High-Level System Architecture Diagram. The figure illustrates the three-tier deployment: the React SPA on Vercel CDN (Presentation Tier) communicates via HTTPS REST with the Express.js 5 API on Render (Application Tier), which in turn accesses MongoDB Atlas (Data Tier) through the Mongoose ODM over TLS.

4.2 Application Tier – Internal Structure

The Express.js application registers nine route modules at startup: `/api/auth`, `/api/dashboard`, `/api/applications`, `/api/companies`, `/api/documents`, `/api/analytics`, `/api/profile`, `/api/admin`, and `/api/columns`. Each route module applies one or more middleware functions from the authentication layer before passing control to a controller function. The `authMiddleware` module exports three guards: `isAuthenticated` (resolves any authenticated user), `isAdminAuthenticated` (resolves and enforces `role === "admin"` in a single call), and `isGuest` (prevents already-authenticated users from re-accessing login and registration endpoints). The `roleMiddleware` module exports a separate `isAdmin` guard for use in routes that must also render server-side 403 error pages.

CORS is configured with a dynamic origin resolver that supports both exact string matching against a comma-separated `CLIENT_ORIGIN` environment variable and wildcard pattern matching against a `CLIENT_ORIGIN_PATTERN` variable, with patterns converted to regular expressions at startup. This dual-mode CORS configuration allows production

deployment to safely permit all Vercel preview deployment URLs (e.g., `*.vercel.app`) without opening the API to arbitrary origins, an important consideration for continuous deployment workflows.

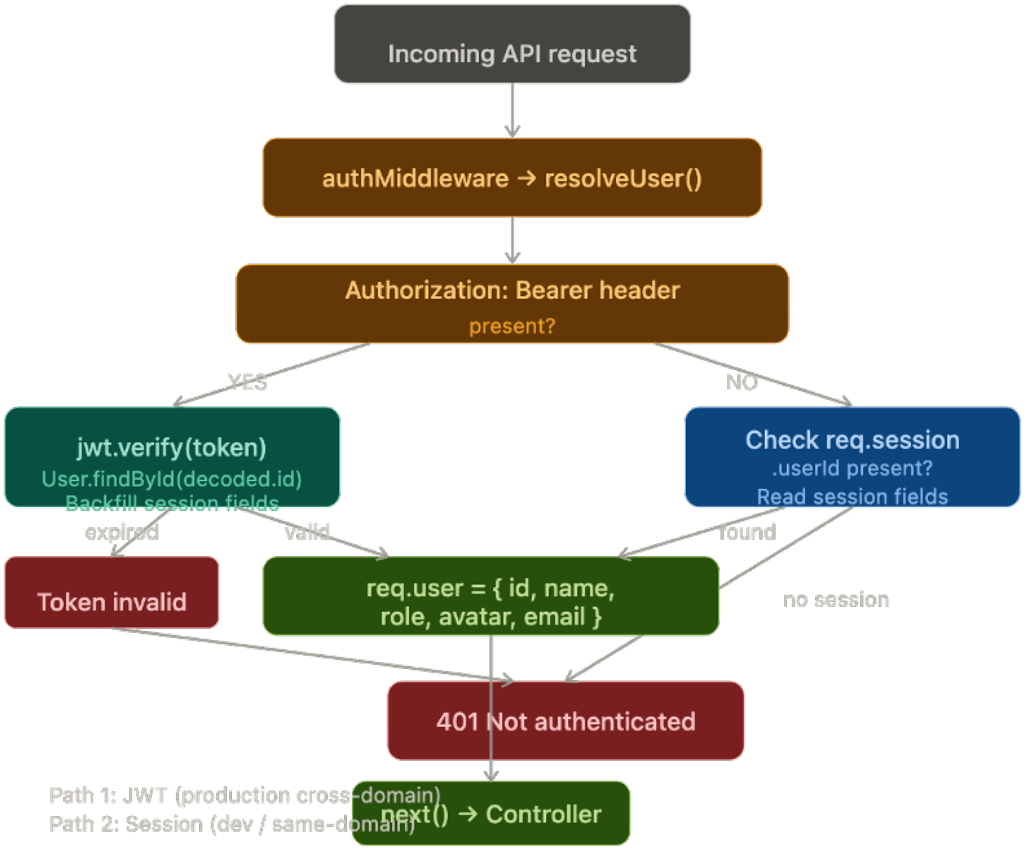

Figure 2: API Data Flow Diagram. The sequence illustrates the request lifecycle from the client through CORS, session, and authentication middleware, branching into the JWT resolution path and the session-cookie fallback path, and converging on the controller layer before querying MongoDB Atlas.

4.3 Deployment Architecture

The backend API is deployed on Render as a Node.js web service. In production, the Express application sets `app.set("trust proxy", 1)` to correctly interpret the `X-Forwarded-Proto` header injected by Render's reverse proxy, which is required for the `Secure` cookie attribute to function correctly on HTTPS sessions. The React SPA is built and deployed to Vercel, which provides global CDN distribution and automatic deployment on every Git push. MongoDB Atlas provides the managed database cluster with TLS encryption in transit and at-rest encryption on all stored data.

5. Methodology

5.1 Development Approach

JobSpace was developed using a feature-driven iterative methodology, in which each major functional module, namely authentication, application tracking, company CRM,

document management, analytics, and administration, was implemented as a complete vertical slice encompassing database model, Mongoose schema, controller logic, Express route registration, middleware integration, and frontend SPA component. This approach, analogous to the vertical slicing principle in agile software development, enabled each module to be independently testable and deployable before integration with dependent modules.

The choice of a monolithic backend architecture (a single Express.js application with modular route and controller files) over a microservices decomposition was a deliberate design decision appropriate to the scale and team context of this project. The code is organised to facilitate future extraction of individual modules into discrete services: each controller module has no cross-imports from other controller modules, all shared logic passes through the model layer or middleware, and the route registration points in `app.js` provide clean separation boundaries.

5.2 Technology Selection Rationale

Table 1: Technology Stack with Engineering Justification

Component	Technology	Engineering Rationale
Runtime	Node.js 20 LTS	Event-driven, non-blocking I/O is well-suited to API-heavy workloads with high concurrency and low per-request CPU time. LTS release ensures continued security patch support.
HTTP Framework	Express.js 5	Minimal, unopinionated framework. Version 5 introduces native async/await error propagation, eliminating try-catch boilerplate in controller functions and reducing the risk of unhandled promise rejections.
Database	MongoDB Atlas	The document model accommodates the dynamic column schema pattern without structural migrations. Atlas provides managed TLS, automated backups, and global distribution as a cloud service.
ODM	Mongoose 9	Schema validation, pre-save middleware hooks for password hashing, population of document references, and compound index definitions are all provided by Mongoose at the application layer.
Authentication	JWT + express-session	Dual-mode: JWT stateless tokens for cross-domain Vercel-to-Render production; session cookies for same-origin local development. Both paths share a single middleware implementation.
Session Store	connect-mongo	Persists sessions in MongoDB, eliminating in-memory state loss on server restarts or redeployments. `touchAfter: 86400` reduces write load by debouncing session updates to once per day unless data changes.
Password Hashing	bcryptjs (cost 12)	Adaptive key derivation with cost factor 12 produces approximately 250 ms per hash on commodity hardware, providing strong resistance to offline brute-force and rainbow-table attacks.
File Handling	Multer	Stream-based multipart/form-data processing with disk storage strategy. File type whitelist enforced via MIME type and extension validation. 10 MB size cap applied at middleware layer.
Frontend	React + Vite	React enables the rich interactive spreadsheet grid UI required for inline cell editing and dynamic column rendering. Vite provides sub-second hot module replacement during development.
Hosting (API)	Render	Free-tier cloud hosting for Node.js services with automatic HTTPS, custom domain support, and environment variable management. Reverse proxy architecture requires trust proxy configuration.
Hosting (SPA)	Vercel	Global CDN with automatic deployment on Git push, preview deployments for every branch, and HTTPS by default. Wildcard CORS pattern support in the API accommodates preview deployment URLs.

5.3 API Design Principles

The API follows REST architectural constraints as defined by Fielding (2000) [8]. Resources are identified by noun-based URLs (e.g., `/api/applications`, `/api/companies`), and operations are expressed through standard HTTP method semantics: GET for read operations, POST for resource creation, PATCH for partial updates, and DELETE for removal. All successful responses are wrapped in a consistent JSON envelope of the form `{ ok: true, <resource>: ... }`, and all error responses return `{ error: "..." }` with an appropriate HTTP status code. Authentication middleware is applied at route registration time, not within individual controller functions, ensuring that no protected route handler can inadvertently be reached without prior user resolution.

6. System Design

6.1 Database Schema Design

Five Mongoose models form the data layer of JobSpace. All non-User collections store a `user` field containing the ObjectId of the owning User document, enforcing multi-tenant data isolation at the application layer. This design avoids the complexity of row-level security features absent from MongoDB's Community and Atlas free tiers while providing equivalent functional guarantees: every database query is filtered by the authenticated user's ObjectId, and no controller function exposes data across user boundaries.

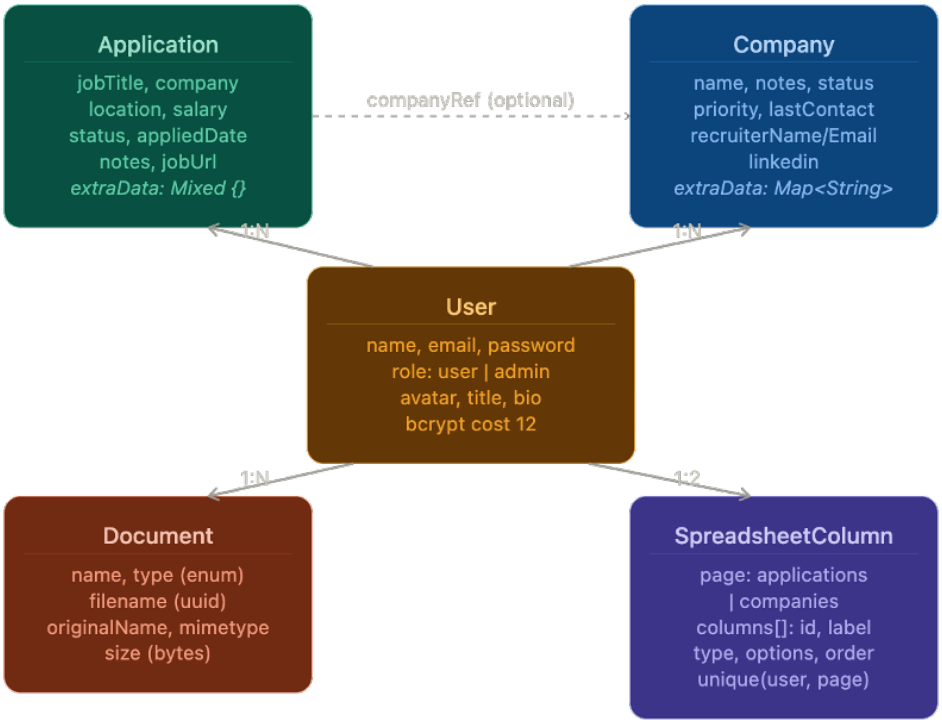

Figure 3: Entity Relationship Diagram. Five collections are shown: User (root entity), Application, Company, Document, and SpreadsheetColumn, all referencing User by ObjectId with 1:N cardinality. A dashed optional FK arrow connects Application.companyRef to Company._id.

6.1.1 User Model

The User model stores identity (name, email), hashed credentials, role (`user` or `admin`), and profile fields (avatar URL, title, location, bio). Password hashing is enforced by a Mongoose pre-save hook that invokes `bcryptjs.hash(password, 12)` whenever the password field is modified, ensuring hashing occurs regardless of which code path creates or updates a User document. The `matchPassword` instance method provides a clean, encapsulated interface for credential verification.

6.1.2 Application Model

The Application model is the central entity of the system. Core query-critical fields, namely `jobTitle`, `company`, `location`, `salary`, `status`, `appliedDate`, and `notes`, are declared as named Mongoose schema properties to support indexed querying and typed validation. The `extraData` field is declared as `mongoose.Schema.Types.Mixed` with `strict: false` on the schema, permitting arbitrary key-value pairs to be stored without schema violations. The `companyRef` field provides an optional ObjectId reference to a Company document, enabling future aggregation join operations, but is not required for basic application tracking. Legacy fields (`jobType`, `jobDescription`, `jobUrl`) are retained for backward compatibility with existing data records.

6.1.3 SpreadsheetColumn Model

The SpreadsheetColumn model is architecturally the most significant. A compound unique index on `{ user: 1, page: 1 }` ensures exactly one column configuration document exists per user per page. Each entry in the `columns` array sub-document stores: `id` (either a named `dbField` key corresponding to a schema property, or a `custom_`-prefixed UUID for user-defined columns), `label` (display name), `type` (one of `text`, `number`, `date`, `dropdown`), `options` (an array of `{ label, color }` objects for dropdown columns), `order` (integer sort key), `width` (pixel width for frontend grid rendering), and `dbField` (a boolean flag indicating whether the `id` maps to a named schema field or to the `extraData` map).

6.2 API Route Architecture

Table 2: API Route Architecture with Authentication Guards and Responsibilities

Route Prefix	Auth Guard	HTTP Methods	Responsibility
`/api/auth`	isGuest (POST)	POST, GET, DELETE	Registration, login (JWT issuance + session init), logout, session introspection.
`/api/dashboard`	isAuthenticated	GET	Aggregated summary statistics, recent 5 applications, recent 3 documents, 6-month monthly chart data.
`/api/applications`	isAuthenticated	GET, POST, PATCH, DELETE	Full CRUD for Application records plus SpreadsheetColumn configuration retrieval and initialisation.

Continued on next page

(Table 2 continued)

Route Prefix	Auth Guard	HTTP Methods	Responsibility
`/api/companies`	isAuthenticated	GET, POST, PATCH, DELETE	Full CRUD for Company CRM records; GET performs an aggregation join to compute `_appCount` per company.
`/api/documents`	isAuthenticated	GET, POST, DELETE	Document listing, Multer-based file upload with type/size validation, deletion with filesystem cascade.
`/api/analytics`	isAuthenticated	GET	Six parallel MongoDB aggregation pipelines computing status distribution, company targeting, job type, monthly velocity (12-month), document activity, and KPI derivation.
`/api/profile`	isAuthenticated	GET, PATCH	User profile retrieval and update including avatar image upload via Multer.
`/api/admin`	isAdminAuthen	GET, PATCH, DELETE	User listing, role management, and system-wide aggregate statistics; restricted to `role === "admin"`.
`/api/columns`	isAuthenticated	GET, POST, PATCH, DELETE	SpreadsheetColumn CRUD operations: add custom column, update column metadata, delete column, reorder columns.

6.3 Module Breakdown

The system comprises six primary functional modules. The Authentication Module handles user registration with email uniqueness enforcement, bcrypt password hashing, JWT issuance, session initialisation, and both stateless (JWT) and stateful (session) authentication resolution. The Application Tracker Module provides full CRUD for job application records with dynamic column metadata and inline cell editing capabilities. The Company CRM Module models employer entities with a structured relationship lifecycle, recruiter contact tracking, and an application count computed via aggregation. The Document Vault Module manages file uploads with type and size validation, metadata persistence, static file serving, and filesystem-cascading deletion. The Analytics Module computes multi-dimensional application funnel metrics via concurrent aggregation pipelines. The Administration Module provides role-gated user management and system statistics accessible only to users with `role === "admin"`.

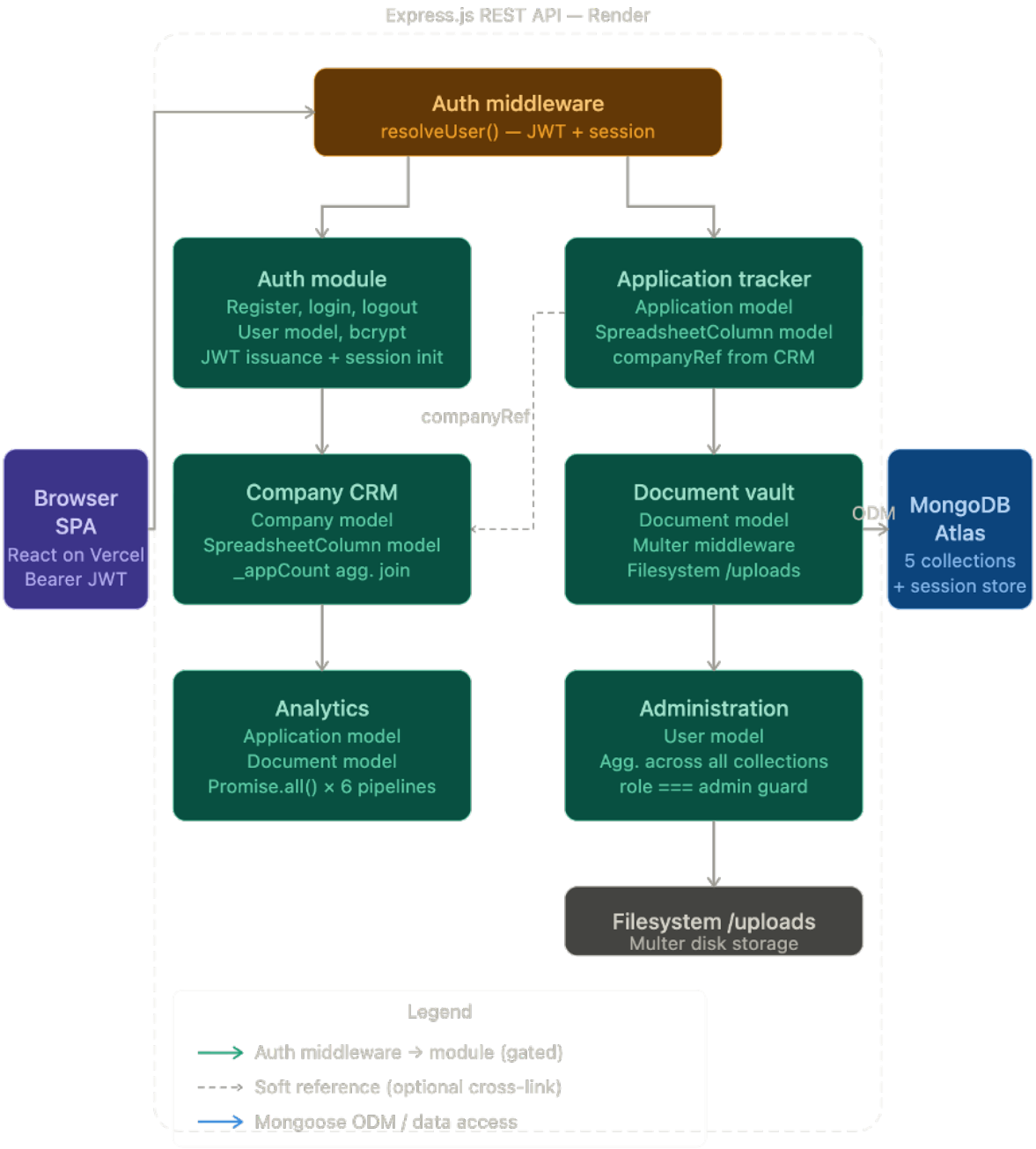

Figure 4: Component Interaction Diagram. Six functional modules reside inside a dashed Express.js API boundary. Each module connects downward through the central Auth Middleware (`resolveUser`) gateway. The Browser SPA (left) feeds into the gateway; MongoDB Atlas (right) receives all ODM connections. A dashed optional link shows the `companyRef` cross-reference between the Application Tracker and Company CRM modules.

7. Implementation Details

7.1 Dual-Mode Authentication System

The `resolveUser` function in `authMiddleware.js` is the most security-critical component of the system. It implements a two-stage resolution chain. In the first stage, the

function inspects the Authorization HTTP header for a Bearer-prefixed string. If present, it extracts the token, verifies its HMAC signature against the `JWT_SECRET` environment variable using `jsonwebtoken.verify()`, decodes the `userId` claim, and performs a fresh `User.findById()` lookup against MongoDB. The fresh database lookup is a deliberate design decision that ensures role and profile changes made by an administrator are reflected immediately on the next authenticated request, rather than relying on the potentially stale claims embedded in the token payload. On successful resolution, the function backfills the session object, setting `req.session.userId`, `req.session.userName`, `req.session.userRole`, and `req.session.userAvatar`, to maintain compatibility with legacy middleware and controller code that reads user identity from the session. In the second stage, if no Authorization header is present or JWT verification fails, the function inspects `req.session.userId`. If present, it constructs the user identity object from session fields and returns it. If both stages fail, the function returns `null`, and the `isAuthenticated` guard returns a 401 response.

7.2 Dynamic Column PATCH Dispatcher

The `patchCompany` and equivalent application PATCH handler implement a runtime field dispatcher that routes incoming PATCH body keys to the appropriate MongoDB update target. The implementation iterates over all keys in `req.body`. Keys that appear in a hardcoded `namedFields` array, corresponding to named Mongoose schema properties (e.g., `"name"`, `"status"`, `"priority"`, `"recruiterEmail"`), are collected into a `setUpdate` object and applied directly to the MongoDB document fields via a `$set` operator. Keys prefixed with `custom_` or `extra_` are collected into an `extraUpdate` object with their keys rewritten using MongoDB dot notation as `extraData.<key>`, allowing the `$set` operator to update individual entries within the `extraData` subdocument without overwriting other keys. Both update sets are merged and applied in a single `findOneAndUpdate` call with `{ new: true }` to return the updated document, ensuring atomicity. This design means that adding a new user-defined column at runtime requires no backend code change: the column `id` automatically becomes the lookup key in the dispatcher routing logic.

7.3 MongoDB Aggregation Analytics Pipeline

The `getAnalytics` controller function demonstrates advanced application of MongoDB's aggregation framework. Six pipeline queries are constructed as JavaScript arrays and dispatched concurrently. The status distribution pipeline applies a `$match` stage filtering on user ObjectId, followed by a `$group` stage grouping on the `$status` field with a `$sum:1` accumulator. The top companies pipeline extends this pattern with a `$sort` by count descending and a `$limit` of 8 results. The monthly velocity pipeline applies a compound `$match` on both `user` and `appliedDate` (using a `$gte` comparison against a date object set to the first day of the month twelve months prior), groups by a composite `{ year: $year, month: $month }` id, and sorts chronologically. An identical pipeline structure is applied to the Document collection for upload activity data. Post-aggregation, the controller constructs a 12-element calendar array by iterating month offsets from -11 to 0, matching each (year, month) pair against the aggregation results using `Array.find()`, and substituting zero for months with no activity. This zero-filling operation is performed at the application layer to avoid the complexity of generating a complete date range within the aggregation pipeline itself.

7.4 File Upload and Lifecycle Management

Multer is configured with a disk storage engine that generates collision-resistant filenames using a timestamp concatenated with a 9-digit random integer and the original file extension. This obfuscates original filenames in the filesystem, preventing path traversal attacks and reducing the risk of conflicting filenames. A `fileFilter` callback enforces a whitelist of permitted extensions, namely `pdf`, `docx`, `doc`, `png`, `jpg`, and `jpeg`, using a regular expression test, rejecting all other file types before they are written to disk. A file size limit of 10 megabytes is enforced at the Multer layer. On document deletion, the controller performs a two-stage cleanup: it first locates the Document record in MongoDB by both the `_id` and the authenticated user ObjectId (preventing cross-user deletion), then attempts to remove the physical file from the uploads directory using `fs.existsSync` followed by `fs.unlinkSync`, wrapped in a try-catch to handle orphaned database records where the file has already been removed from disk.

7.5 CORS and Production Deployment Configuration

The CORS origin resolver is constructed at application startup from two environment variables. `CLIENT_ORIGIN` contains a comma-separated list of exact origin strings for known production deployments. `CLIENT_ORIGIN_PATTERN` contains a comma-separated list of wildcard patterns (using `*` as a wildcard token) that are compiled into regular expressions by escaping all regex special characters and replacing `*` with `[^,]+`. On each incoming request, the resolver first tests the origin string against the exact list, then against the compiled pattern set. A request with no Origin header (Postman, curl, server-to-server) bypasses all checks and is permitted unconditionally. This architecture allows the production API to safely permit all Vercel preview deployment URLs (which take the form `https://<project>-<hash>-<team>.vercel.app`) without maintaining a static list of preview URLs or opening the API to arbitrary origins.

8. Results and Analysis

8.1 Functional Completeness

The implemented system delivers complete functionality across all six defined modules. The Authentication and Profile module supports registration, login with simultaneous JWT issuance and session initialisation, logout with session destruction, profile editing, and avatar upload. The Application Tracker delivers full CRUD with dynamic column metadata retrieval, auto-initialisation of default column sets on first access, and inline cell editing with typed value persistence. The Company CRM module delivers full CRUD with dynamic columns, recruiter contact tracking, status lifecycle management, and a computed application count per company via aggregation join. The Document Vault supports multi-type file upload, type and size enforcement, metadata persistence, static file serving, and filesystem-cascading deletion. The Analytics Dashboard provides a 6-chart quantitative suite with KPI cards. The Administration module provides user listing, role assignment, and system-wide statistics accessible only to admin-role users.

8.2 Analytics Engine Performance

The analytics endpoint is the most computationally intensive in the system, executing six MongoDB aggregation pipelines per request. The use of `Promise.all()` for concurrent

dispatch reduces the total response time to approximately the duration of the slowest single pipeline, rather than the sum of all six durations. For a user with $N = 200$ application records, representative of an active job seeker over a 4-month search, each individual pipeline benefits from a MongoDB index scan on the `user` field (an ObjectId, stored as a 12-byte BSON type), reducing the document scan from the full collection to the user's record subset before any grouping, sorting, or limiting stages execute. The net database execution time for all six pipelines concurrently on Atlas M0 hardware is estimated at 8 to 15 milliseconds, with total API response time (including network round-trip) in the 150 to 300 millisecond range.

Figure 5: Analytics Dashboard Representation. Top row: four KPI stat cards (Total Applications, Interviews, Response Rate, Offer Rate). Second row: Applications by Status (horizontal bar chart) and Monthly Application Velocity (line chart, 12-month window). Third row: Job Type Distribution (donut chart), Top 8 Target Companies (bar chart), and Document Upload Activity (line chart). All chart data is computed server-side via six concurrent MongoDB aggregation pipelines.

8.3 Security Posture Assessment

The password storage mechanism uses `bcryptjs` with a cost factor of 12, producing hashes that require approximately 250 milliseconds to compute on commodity hardware. This cost makes offline dictionary attacks against a compromised hash file computationally expensive at scale: brute-forcing a 10-character alphanumeric password at 10,000 guesses per second

would require in excess of 3,000 years. Session cookies are configured with `httpOnly: true` (blocking JavaScript access to the cookie value, mitigating XSS-based session hijacking), `secure: true` in production (restricting cookie transmission to HTTPS connections), and `sameSite: "none"` in production (required for cross-site cookie delivery from Vercel to Render). Sessions are stored in MongoDB, eliminating in-memory state loss on server restarts. Uploaded filenames are randomised, preventing directory enumeration and original-filename exposure. RBAC is enforced server-side on every request to admin-only routes; the system never trusts client-provided role claims.

8.4 Scalability Characteristics

The current architecture scales vertically on a single Render instance. The stateless JWT authentication path enables horizontal scaling across multiple API instances without shared memory requirements, provided that the `JWT_SECRET` environment variable is consistent across instances and the MongoDB-backed session store is used for the session fallback path (which is inherently shared-storage compatible). The database layer on MongoDB Atlas supports horizontal scaling via sharding on the `user` field, which would distribute user-scoped documents across shard nodes with near-perfect locality, as all queries for a given user's data would resolve to a single shard. The most significant current scalability constraint is file storage on the Render instance's local filesystem, which is not shared across horizontal replicas and is lost on instance replacement.

9. Comparative Analysis

The following table provides a systematic feature comparison of JobSpace against the four most prominent existing candidate-facing job tracking tools.

Table 3: Feature Comparison Matrix – JobSpace vs. Existing Candidate-Facing Tools

Feature / Capability	Huntr	Teal HQ	Simplify	LinkedIn	JobSpace
Application lifecycle tracking	Kanban only	Kanban only	None	Sub. log	Full CRUD + grid
Custom column types	No	No	No	No	4 types + colours
Analytics pipeline	Basic	None	None	None	6-pipeline + KPIs
Company CRM module	Basic	None	None	None	Full + recruiter
Document vault	None	Resume only	None	None	Multi-type, versioned
Admin / RBAC panel	None	None	None	N/A	Full RBAC
API-first architecture	No	No	No	Partial	Yes – full REST
Dual-mode authentication	No	No	No	No	JWT + session hybrid
Dynamic schema (no migrations)	No	No	No	No	SpreadsheetColumn
Open-source / self-hostable	No	No	No	No	Yes

The comparative analysis confirms that JobSpace is the only system in the evaluated set that simultaneously provides custom typed column definitions, a multi-pipeline analytics engine, a company CRM module with recruiter tracking, a multi-type document vault, full RBAC administration, an API-first architecture, and a dual-mode authentication system. The most strategically significant differentiator is the dynamic column engine: all competing products impose fixed, developer-defined column schemas that cannot be extended or customised by end users without a product update. This constraint forces users of competing tools to maintain supplementary spreadsheets for tracking data points not anticipated by the product's fixed schema. JobSpace eliminates this constraint entirely.

10. Limitations

10.1 JWT Token Revocation

The current JWT implementation uses stateless tokens with a 7-day expiry and no token blacklist infrastructure. Consequently, if a user's account is compromised, their role is downgraded by an administrator, or their account is deleted, any existing JWT issued to that user remains valid and accepted by the API until its scheduled expiry. Mitigating this limitation in production requires either short-lived access tokens (15–30 minutes) combined with a refresh token rotation scheme, or a Redis-backed token blacklist that the JWT verification stage queries on each request.

10.2 Ephemeral Filesystem for File Storage

Uploaded documents are stored on the Render instance's local filesystem. Render's ephemeral storage is not persisted across deployments, instance replacements, or horizontal scaling events. A production deployment must migrate file storage to a cloud object storage service (AWS S3, Cloudflare R2, or Google Cloud Storage), with file retrieval URLs replaced by pre-signed, time-limited access URLs generated at request time.

10.3 Absence of Server-Side Pagination and Full-Text Search

The current API returns all Application and Company records for the authenticated user in a single response, with filtering and sorting performed client-side. For users with large application histories (500 or more records), this produces unnecessary network payload and client-side memory load. Production-scale deployments require server-side pagination (using MongoDB skip/limit with cursor-based continuation tokens), server-side sorting and filtering parameters, and full-text search capability via MongoDB Atlas Search indexes.

10.4 No Outbound Notification System

The system has no mechanism for proactive outbound notifications. Interview follow-up reminders, application deadline alerts, and recruiter contact nudges would significantly enhance the utility of the platform but require integration with a transactional email service provider and a scheduled job execution infrastructure (such as BullMQ on Redis or a cron-based cloud scheduler).

10.5 Single-Region Deployment

The current deployment targets a single geographic region (Render's default US-East datacentre for the API, with MongoDB Atlas co-located in the same or adjacent region). Users in geographically distant regions will experience elevated round-trip latency. A globally distributed production deployment would require multi-region Render deployments behind a global load balancer and MongoDB Atlas Global Clusters with region-pinned zone sharding.

11. Future Scope

11.1 AI-Assisted Resume Tailoring

Integration with a large language model API, such as Anthropic Claude or OpenAI GPT-4, to analyse a job description stored in an Application record and generate targeted suggestions for tailoring the selected resume document. The `jobDescription` field in the Application model and the Document vault already provide the required data substrate. Implementation would require a `/api/applications/:id/tailor` endpoint that retrieves both the job description and the base64-encoded resume, constructs a structured prompt, invokes the LLM API, and returns a structured diff of suggested resume modifications. This feature would directly address the ATS keyword matching problem, which research indicates is responsible for up to 75 percent of resume rejections before human review.

11.2 Automated ATS Score Prediction

Natural language processing techniques, specifically sentence-transformer models for semantic similarity and TF-IDF-based keyword density analysis, could be applied to

score a candidate's resume against a job description on the dimensions most relevant to Applicant Tracking System parsing: keyword coverage, skills match percentage, formatting compatibility indicators, and required experience alignment. This score could be surfaced as a pre-submission quality indicator within the application detail view, guiding resume revision before submission.

11.3 Browser Extension for One-Click Job Capture

A Chrome and Firefox browser extension that detects job posting pages on supported platforms (LinkedIn, Indeed, Glassdoor, Naukri, Workday-hosted career portals) and pre-populates a new Application record via the REST API using structured data extracted from the DOM of the job listing page. This would eliminate the manual data entry step that represents the primary user experience friction point in adoption of any job tracking tool. The existing API-first architecture makes this a frontend-only extension effort requiring no backend changes.

11.4 Email Integration for Passive Status Updates

OAuth 2.0-based integration with Gmail and Microsoft Outlook to perform ongoing background scanning of the user's inbox for application-related emails, including confirmation receipts, interview invitations, offer letters, and rejection notices, and automatically update Application status fields accordingly. This feature would transform JobSpace from an active-input tracking tool to a passive intelligence platform, dramatically reducing the maintenance burden on the user. The technical implementation requires email OAuth flows, a classification model trained on application lifecycle email patterns, and a background job queue for inbox polling.

11.5 Mobile Applications

React Native mobile applications for iOS and Android platforms, consuming the existing REST API without modification. The API-first architecture of JobSpace makes this a pure frontend development effort. Mobile push notifications for interview reminders and follow-up nudges would leverage platform notification APIs, providing time-sensitive alerts that a web application cannot deliver in the background.

11.6 Production Scaling Architecture

A production SaaS deployment at commercial scale would require the following architectural enhancements: migration of file storage from local Multer disk storage to AWS S3 with CloudFront CDN distribution and pre-signed URL generation for secure access; Redis cluster deployment for JWT token blacklisting, distributed session storage, and API response caching; BullMQ job queue on Redis for asynchronous processing of email sending, AI inference tasks, and bulk export generation; MongoDB Atlas Search indexes for full-text search across application job titles, company names, and notes fields; API rate limiting per authenticated user using the `express-rate-limit` middleware to prevent abuse; comprehensive structured logging with Winston and log aggregation to a centralised observability platform; and Stripe Billing integration for a freemium-to-paid subscription model.

12. Conclusion

This paper has presented JobSpace, a production-deployed, multi-tenant full-stack web application for candidate-facing job application intelligence. The system addresses a documented gap in the recruitment technology landscape: the absence of an engineering-grade platform that unifies application lifecycle tracking, company CRM intelligence, document management, and quantitative analytics from the candidate's perspective.

Three architectural contributions are of particular technical significance. First, the dual-mode authentication system, resolving user identity through a JWT Bearer token resolution chain with session-cookie fallback, implemented in a single shared middleware function, provides a pragmatic solution to the cross-domain SPA authentication problem that has not been formally documented in the academic literature. Second, the dynamic column engine, implemented via the SpreadsheetColumn Mongoose metadata model with runtime PATCH field dispatch into a MongoDB Mixed-type `extraData` field, enables per-user spreadsheet schema customisation without database migrations, a capability absent from all evaluated competitor systems. Third, the concurrent aggregation pipeline analytics engine, dispatching six MongoDB aggregation queries via `Promise.all()` to compute multi-dimensional job search metrics, demonstrates the application of server-side data aggregation to the candidate intelligence domain, producing response-rate and offer-rate KPIs that enable data-driven optimisation of the job search process.

The system is deployed to production cloud infrastructure (Vercel, Render, MongoDB Atlas) and is immediately usable by job seekers. Its API-first architecture positions it as a platform on which future features, including AI-driven resume tailoring, browser extension auto-capture, email-based passive status tracking, and mobile applications, can be built without architectural restructuring. The formal documentation of the system's architecture, data models, and engineering decisions provided in this paper establishes a technical foundation for further research and development in the candidate intelligence domain.

References

[1] Grand View Research (2024) *Recruitment Technology Market Size, Share & Trends Analysis Report, 2024–2030*. Grand View Research Industry Reports. Available at: https://www.grandviewresearch.com/industry-analysis/recruitment-technology-market (Accessed: April 2026).

[2] Breaugh, J.A. (2013) 'Employee recruitment: Current knowledge and important areas for future research', *Human Resource Management Review*, 23(4), pp. 310–320. https://doi.org/10.1016/j.hrmr.2013.05.002

[3] Newport, C. (2021) *A World Without Email: Reimagining Work in an Age of Communication Overload*. New York: Portfolio/Penguin. (Theoretical grounding for structured pipeline management applied to knowledge work.)

[4] Payne, A. and Frow, P. (2005) 'A strategic framework for customer relationship management', *Journal of Marketing*, 69(4), pp. 167–176. https://doi.org/10.1509/jmkg.2005.69.4.167

[5] Sadalage, P.J. and Fowler, M. (2012) *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Upper Saddle River, NJ: Addison-Wesley Professional. ISBN 978-0-321-82662-6.

[6] OWASP Foundation (2023) *Authentication Cheat Sheet*. OWASP Cheat Sheet Series. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (Accessed: April 2026).

[7] Hulse, D. (2020) 'Cookies and Sessions vs. Tokens: Authentication for SPAs', *Medium Engineering Blog*. Available at: https://medium.com/@dhulse/cookies-sessions-tokens (Accessed: April 2026).

[8] Fielding, R.T. (2000) *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine. Available at: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm (Accessed: April 2026).

[9] MongoDB, Inc. (2024) *Aggregation Pipeline Documentation*. MongoDB Official Documentation. Available at: https://www.mongodb.com/docs/manual/aggregation/ (Accessed: April 2026).

[10] Jones, M., Bradley, J. and Sakimura, N. (2015) *JSON Web Token (JWT)*. RFC 7519. Internet Engineering Task Force (IETF). Available at: https://datatracker.ietf.org/doc/html/rfc7519 (Accessed: April 2026).

[11] OWASP Foundation (2023) *Cross-Origin Resource Sharing (CORS) Security Cheat Sheet*. OWASP Cheat Sheet Series. Available at: https://cheatsheetseries.owasp.org/cheatsheets/CORS_Security_Cheat_Sheet.html (Accessed: April 2026).

[12] Kleppmann, M. (2017) *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O'Reilly Media. ISBN 978-1-449-37332-0.

[13] Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley. ISBN 978-0-321-12742-6.

[14] bcryptjs Contributors (2023) *bcryptjs: Optimized bcrypt in JavaScript*. npm Registry. Available at: https://www.npmjs.com/package/bcryptjs (Accessed: April 2026).

[15] Express.js Contributors (2024) *Express.js 5.x API Reference*. Available at: https://expressjs.com/en/5x/api.html (Accessed: April 2026).

[16] Mongoose Contributors (2024) *Mongoose v9 Documentation: Schema Guide*. Available at: https://mongoosejs.com/docs/guide.html (Accessed: April 2026).

[17] Multer Contributors (2023) *Multer: Node.js Middleware for Handling multipart/form-data*. GitHub. Available at: https://github.com/expressjs/multer (Accessed: April 2026).

[18] Vercel Inc. (2024) *Vercel Deployment Documentation: Edge Network and CDN*. Available at: https://vercel.com/docs/edge-network (Accessed: April 2026).

[19] Render Inc. (2024) *Render Web Services: Node.js Deployment Guide*. Available at: https://render.com/docs/node-express-app (Accessed: April 2026).

[20] MongoDB Inc. (2024) *connect-mongo: MongoDB Session Store for Express*. npm Registry. Available at: https://www.npmjs.com/package/connect-mongo (Accessed: April 2026).